

Week 1 – Getting Started with Linux and the Command Line

Topics covered

- RH124 Chapter 1: Getting Started on Red Hat Enterprise Linux
- RH124 Chapter 2: Accessing the Command Line
- Lab: Access the Command Line

The main purpose of Week 1 is to understand how the Linux shell works, how commands are structured, and how to view basic system and file information using the terminal.

Week 1 essential commands

Command	Purpose
<code>whoami</code>	Shows the current username
<code>hostname</code>	Shows the computer's hostname
<code>date</code>	Shows the date and time
<code>passwd</code>	Changes a password
<code>ls</code>	Lists directory contents
<code>ls -l</code>	Displays a detailed listing
<code>ls -r</code>	Reverses the listing
<code>type</code>	Identifies the source of a command
<code>file</code>	Identifies a file type
<code>cat</code>	Displays complete file contents
<code>less</code>	Views a long file page by page
<code>head</code>	Displays the beginning of a file
<code>tail</code>	Displays the end of a file
<code>wc</code>	Counts lines, words and bytes

`history` Displays previously used
 commands

Main point to remember: Linux commands follow the structure `command option argument`.
The command specifies the action, the option changes how it operates, and the argument specifies what it acts on.

1. Understanding the shell

A shell allows the user to communicate with the operating system. It accepts commands, interprets them and passes the required instructions to the Linux kernel.

There are two main ways to interact with an operating system:

- **Graphical User Interface (GUI):** Uses windows, icons and menus.
- **Command Line Interface (CLI):** Uses typed commands in a terminal.

Linux commonly uses the **Bash shell**, which stands for **Bourne Again Shell**.

The general structure of a Linux command is:

`command [options] [arguments]`

Example:

`ls -l /etc`

In this command:

- `ls` is the command.
- `-l` is an option.
- `/etc` is the argument.

Linux commands are case-sensitive. Therefore:

`ls`

is not the same as:

`LS`

2. Types of commands

Linux commands can come from several sources.

Command type	Meaning
Internal command	Built directly into the shell, such as <code>cd</code>
External command	A separate executable file, such as <code>ls</code>
Alias	A customised shortcut for another command
Function	A collection of commands grouped under one name

You can use the following command to determine the source of a command:

```
type command_name
```

Examples:

```
type cd
```

```
type ls
```

Possible output:

```
cd is a shell builtin
```

```
ls is aliased to 'ls --color=auto'
```

3. Using commands, options and arguments

Running `ls` without an argument lists the contents of the current directory:

```
ls
```

Running it with `-l` provides a detailed listing:

```
ls -l
```

The detailed output includes information such as:

- Permissions
- File owner
- Group owner
- File size

- Modification date
- File or directory name

Options can sometimes be combined:

```
ls -l -r
```

This produces the same result as:

```
ls -lr
```

Here:

- `-l` displays detailed information.
- `-r` reverses the order of the output.

Some short options also have long versions:

```
ls -r
```

```
ls --reverse
```

Both commands reverse the listing order.

An argument tells the command what object to work on:

```
ls /etc/ppp
```

This lists the contents of `/etc/ppp` instead of the current directory.

A command may also accept several arguments:

```
ls /etc/ppp /etc/ssh
```

This displays the contents of both directories.

4. Accessing the terminal

A terminal is a text-based interface used to enter commands and view their output.

In a graphical Linux environment, the terminal can usually be opened through the applications menu.

Linux also supports multiple virtual consoles. The notes describe switching between them with:

```
Ctrl + Alt + F1
```

Ctrl + Alt + F2
Ctrl + Alt + F3 to F6

Each virtual console can provide a separate login session.

The exact key that returns to the graphical interface may depend on the Linux version and virtual-machine configuration.

5. Running multiple commands

Normally, each command is entered on a separate line. A semicolon can be used to run multiple commands on one line:

```
date; whoami; hostname
```

The shell runs the commands from left to right.

The semicolon does not check whether the previous command succeeded. It simply separates the commands.

6. Displaying the date and time

Use:

```
date
```

This displays the current system date, time and timezone.

A format string can be used to control the output:

```
date "+%Y-%m-%d"
```

Example output:

```
2026-07-23
```

Another example:

```
date "+%H:%M:%S"
```

Example output:

```
17:30:45
```

Common date formatting symbols:

Symbol	Meaning
%Y	Four-digit year
%m	Month
%d	Day
%H	Hour
%M	Minute
%S	Second

7. Changing a password

The `passwd` command changes a user's password.

To change your own password:

```
passwd
```

You will normally be asked to enter:

1. Your current password
2. Your new password
3. The new password again

The root user can change another user's password:

```
passwd username
```

Example:

```
passwd student
```

A normal user generally cannot change another user's password.

8. Identifying file types

Linux does not depend entirely on file extensions such as `.txt` or `.jpg` to identify a file.

Use:

file filename

Examples:

file /etc/passwd

file /bin/ls

Possible output:

/etc/passwd: ASCII text

/bin/ls: ELF 64-bit executable

The **file** command examines the contents or header of the file to determine its type.

9. Viewing file contents with **cat**

To display the complete contents of a file:

cat filename

Example:

cat /etc/hostname

To display multiple files together:

cat file1 file2

The word **cat** comes from **concatenate**, meaning to join content together.

cat is suitable for short files. A large file may scroll past the screen too quickly.

10. Viewing long files with **less** and **more**

For long files, use:

less filename

Example:

less /etc/passwd

Useful controls inside `less`:

Key	Action
Spacebar	Move forward one page
<code>b</code>	Move back one page
Up/Down arrows	Move one line
<code>/word</code>	Search for text
<code>n</code>	Move to the next search result
<code>h</code>	Display help
<code>q</code>	Quit

The notes also mention:

`more filename`

However, `less` normally provides more navigation features than `more`.

11. Viewing the beginning of a file

The `head` command displays the beginning of a file.

`head filename`

By default, it displays the first ten lines.

To display the first five lines:

`head -5 filename`

or:

`head -n 5 filename`

Example:

`head -n 5 /etc/passwd`

12. Viewing the end of a file

The `tail` command displays the end of a file.

```
tail filename
```

By default, it displays the final ten lines.

To display the final five lines:

```
tail -5 filename
```

or:

```
tail -n 5 filename
```

Example:

```
tail -n 5 /etc/passwd
```

A commonly used additional option is:

```
tail -f logfile
```

This continuously displays new lines added to a file. It is useful when monitoring log files. This specific use is general Linux knowledge rather than a stated Week 1 lab requirement.

13. Counting lines, words and characters

Use the `wc` command:

```
wc filename
```

The output normally contains:

```
lines words bytes filename
```

Common options:

Command	Meaning
<code>wc -l filename</code>	Counts lines

<code>wc -w</code>	Counts words
<code>filename</code>	
<code>wc -c</code>	Counts bytes
<code>filename</code>	
<code>wc -m</code>	Counts
<code>filename</code>	characters

Example:

```
wc -l /etc/passwd
```

This can be used to estimate how many account entries are stored in `/etc/passwd`, because each account normally uses one line.

14. Tab completion

Pressing the **Tab key** allows Bash to complete command names and file paths.

Example:

```
cat /etc/hos
```

Press **Tab**. Bash may complete it as:

```
cat /etc/hosts
```

When several matches exist, press **Tab** twice to display the possible choices.

Tab completion helps to:

- Reduce typing
 - Prevent spelling mistakes
 - Discover available commands and files
-

15. Command history

Bash keeps a record of commands previously entered.

Use:

history

To display only the most recent five entries:

history 5

Use the Up and Down arrow keys to cycle through previous commands.

Use:

Ctrl + R

to search command history.

After pressing **Ctrl + R**, type part of an earlier command. Bash displays a matching command.

To clear the current history list:

history -c

Bash stores saved command history in:

~/.bash_history

The history is normally written to this file when the shell session ends.

16. Week 1 lab solution flow

The lab is mainly testing whether you can access the command line and use basic Bash commands correctly.

A suitable solution sequence is:

whoami
hostname
date

Purpose:

- **whoami** confirms the current user.
- **hostname** displays the system name.
- **date** confirms the current system date and time.

List the current directory:

ls

Display a detailed list:

ls -l

Reverse the output:

ls -lr

List a specific directory:

ls /etc

Identify different file types:

file /etc/passwd

file /bin/ls

View a short file:

cat /etc/hostname

View a longer file:

less /etc/passwd

Press **q** to exit.

View the first five lines:

head -n 5 /etc/passwd

View the last five lines:

tail -n 5 /etc/passwd

Count the number of lines:

wc -l /etc/passwd

Check previous commands:

history

The expected result is that each command executes successfully and displays the appropriate user, system, date or file information. The actual output may differ between machines because usernames, hostnames, files and account counts may not be identical.

17. Common mistakes

Using capital letters

Incorrect:

LS

Correct:

ls

Linux commands are case-sensitive.

Forgetting spaces

Incorrect:

ls-l

Correct:

ls -l

The command, options and arguments must normally be separated by spaces.

Using the wrong dash

Correct short option:

ls -l

Correct long option:

ls --reverse

A short option uses one hyphen, while a long option usually uses two.

Trying to use **cat** on a very large file

Use:

less filename

instead, so that the file can be read one page at a time.

Not knowing how to exit **less**

Press:

q
